BookMyShow

Low-Level Design Guide

Movie ticket booking system with a C++17 booking engine, Node.js API, React client and Supabase

What this guide covers	
Project overview	Architecture, request flow, folder layout
Low-level design	Every class in the C++ core, with the real code
Design patterns	Singleton, Strategy, Factory, Observer, Facade, State, Aggregate, Dependency Injection
Concurrency	How double booking is prevented, why it cannot deadlock
Interview prep	2-minute pitch, 10-minute walkthrough, 15 likely questions

Stack: C++17 (CMake, GoogleTest, nlohmann/json, Node-API) | Node.js 20 + Express 5 | React 19 + Vite 6 | Supabase Postgres

Contents

#	Section	Contents
1	Project overview	What the app does, architecture, folder layout
2	Domain model	Entities and how they relate
3	Design patterns at a glance	Every pattern, where it is, why it is there
4	Class-by-class walkthrough	Seat, Show, lock manager, pricing, payments, booking, refunds, notifications, facade
5	Concurrency design	Preventing double booking without deadlocks
6	End-to-end booking flow	One request from click to database
7	Backend and persistence	Hydration, compensation, database guard, security
8	SOLID principles in this code	Where each principle shows up
9	Testing	What is tested and how
10	Trade-offs and scaling	Limitations and how to grow the design
11	Interview guide	Pitch, walkthrough and likely questions
12	Appendix	File index and how to run the project

1. Project overview

BookMyShow is a movie ticket booking system. A user picks a city, a movie and a showtime, selects seats on a live seat map, and those seats are **held for 5 minutes** while the user pays. If payment succeeds the seats become **booked**; if the timer runs out they are **released** automatically. Confirmed bookings can be cancelled with a refund that depends on how close the show is.

The heart of the project, and the subject of this guide, is the **booking engine**: a C++17 library in `core/` that contains all the business rules. The Node.js backend and the React frontend are thin layers around it.

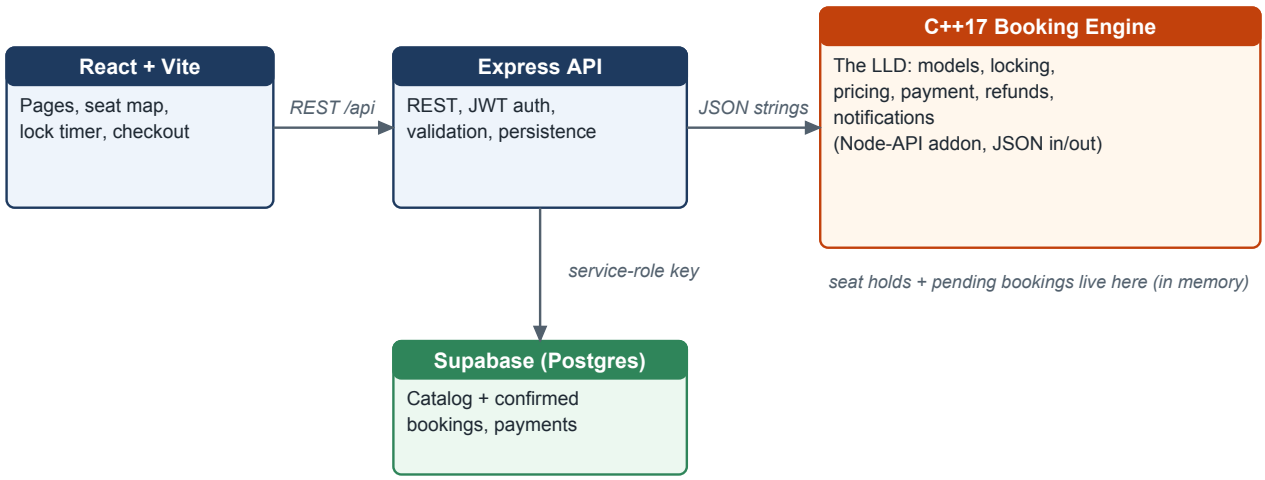


Figure 1 - System architecture

Who owns what

Layer	Responsibility	Key files
C++ core (the LLD)	Seat states, locking with TTL, pricing rules, payment strategies, booking lifecycle, refunds, notifications	core/include, core/src
Node-API binding	Exposes BookingEngine to JavaScript as JSON-in/JSON-out calls	core/src/bindings/addon.cpp, backend/src/engine/engine.js
Express backend	REST API, JWT auth, input validation, loading shows from the database into the engine, persisting confirmed bookings	backend/src/
React frontend	Browsing, seat map with polling, lock countdown, checkout, booking history	frontend/src/
Supabase (Postgres)	Catalog data and confirmed bookings/payments. Row Level Security blocks public access	db/schema.sql, db/seed.sql

State rules

- **Seat holds and pending bookings live only in C++ memory**, protected by mutexes and a TTL (default 300 s). They are short-lived, so a database round trip per click is unnecessary.
- When a show is first requested, the backend **hydrates** it: it loads seats and confirmed bookings from Postgres into the engine.
- Only after the engine confirms a payment does the backend write the booking, its seats and the payment to Postgres.

- Postgres has `UNIQUE (show_id, seat_id)` on booked seats as a final guard against double booking.

Folder layout of the core

```
core/
  include/                    (headers)            src/  (same layout, .cpp files)
    BookingEngine.hpp         Facade               BookingEngine.cpp
    enums/Enums.hpp           SeatType, SeatStatus, BookingStatus, PaymentMode, PaymentStatus
    models/                   City, Theatre, Screen, Seat, Movie, User,
                              ShowSeat (seat state machine), Show (aggregate root),
                              Booking (booking state machine), Payment
    services/                 SeatLockManager (Singleton), BookingService, NotificationService
    pricing/                  PricingStrategy + SeatType/Weekend/PeakHour rules, PricingEngine
    payment/                  PaymentStrategy + Card/Upi/NetBanking, PaymentFactory
    refund/                   RefundPolicy + TieredRefundPolicy, FullRefundPolicy
    observer/                 BookingObserver + EmailNotifier, SmsNotifier
  src/bindings/addon.cpp      Node-API wrapper
  src/main.cpp                Runnable demo
  tests/                      GoogleTest suites (20 tests)
```

2. Domain model

The physical world (cities, theatres, screens, seats) is separated from what is sold (a seat *for one show*). That separation is the key modelling decision: the same physical seat A5 is booked independently for the 10:00 show and the 19:00 show.

Entity	Represents	Notes
City -> Theatre -> Screen -> Seat	Physical venue hierarchy	Seat has a row, number and SeatType (REGULAR, PREMIUM, VIP)
Movie	Film metadata	Title, duration, language, genre
Show	One screening of a movie on a screen at a time	Aggregate root: owns its ShowSeats and the lock that guards them
ShowSeat	A seat for one specific show	State machine AVAILABLE / LOCKED / BOOKED with a TTL hold
Booking	A user's purchase of seats for a show	State machine PENDING / CONFIRMED / CANCELLED / EXPIRED
Payment	Money movement for a booking	Mode + status; refunds are separate rows
User	Customer	Auth handled by the backend (bcrypt + JWT)

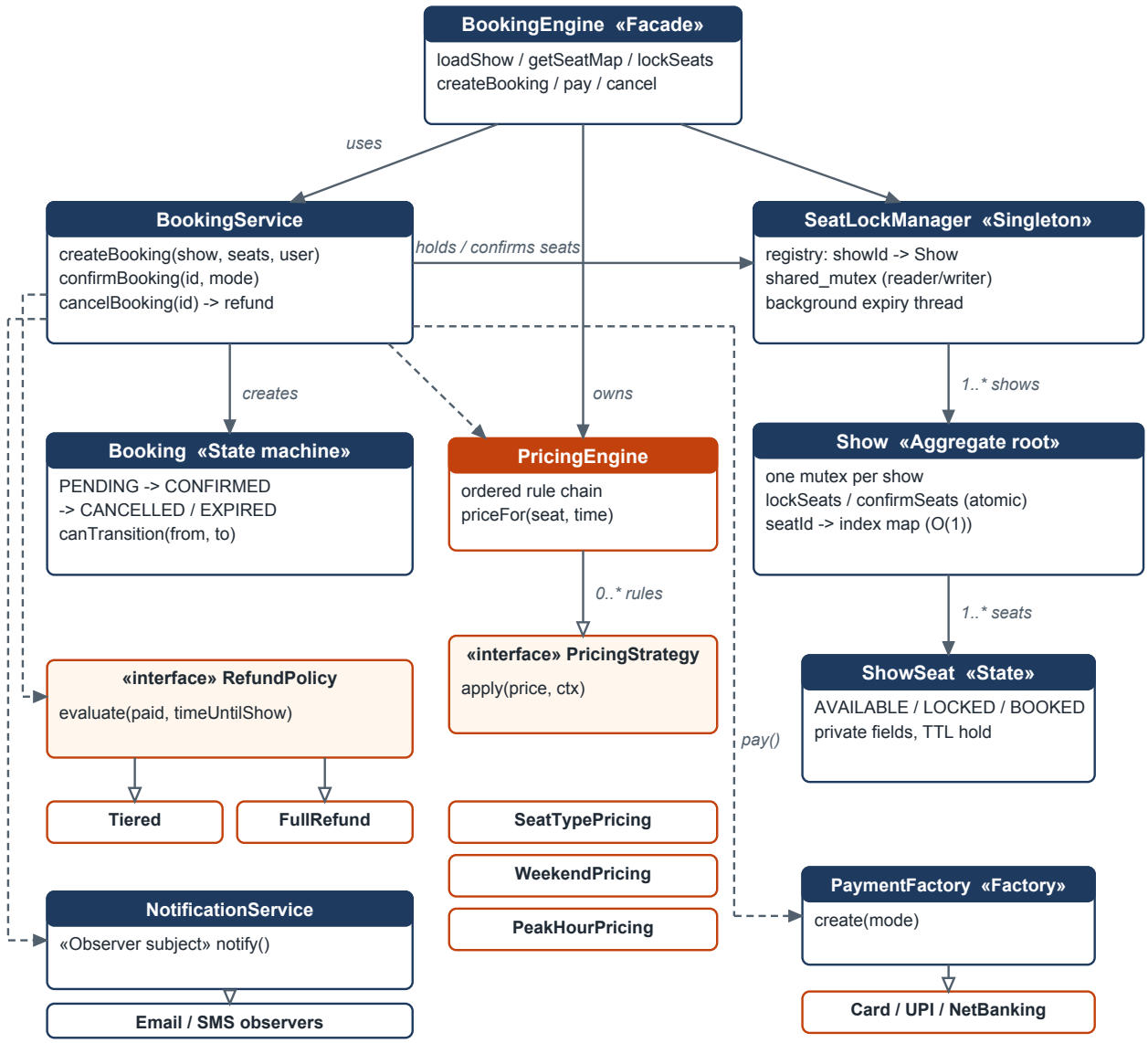


Figure 2 - Class diagram of the booking engine (hollow arrow = implements, dashed = uses)

3. Design patterns at a glance

Pattern	Where	Why it is used here
Singleton	SeatLockManager::getInstance()	Exactly one registry of live shows and one expiry thread per process, so every request sees the same seat holds. Thread-safe via a function-local static.
Strategy (pricing)	PricingStrategy -> SeatTypePricing, WeekendPricing, PeakHourPricing	Each pricing rule is an independent, swappable class.
Chain of rules	PricingEngine	Applies an ordered list of strategies; a new rule is added without editing existing code (Open/Closed).
Strategy (payment)	PaymentStrategy -> CardPayment, UpiPayment, NetBankingPayment	Each payment mode has its own pay() behind one interface.
Factory	PaymentFactory::create(PaymentMode)	Callers ask for a mode; they never construct concrete payment classes.
Strategy (refunds)	RefundPolicy -> TieredRefundPolicy, FullRefundPolicy	Cancellation rules change by business decision, not by editing BookingService.
State machine	ShowSeat, Booking	Legal transitions are explicit; illegal ones (e.g. EXPIRED -> CONFIRMED) are rejected.
Observer	NotificationService + BookingObserver (Email, SMS)	Booking logic announces events; notifiers subscribe without the service knowing them.
Facade	BookingEngine	One small JSON API hides all classes from Node.js.
Aggregate root (DDD)	Show	All seat changes go through Show under one lock, keeping multi-seat invariants intact.
Dependency injection	BookingService constructor	Pricing engine and refund policy are passed in, so tests and products can swap them.
RAII	std::lock_guard, std::shared_lock, smart pointers	Locks and memory are released automatically, even on early return.

4. Class-by-class walkthrough

Each part below follows the same shape: **what problem it solves**, **the pattern**, **the real code**, and **what to say in an interview**.

4.1 ShowSeat - the seat state machine

A ShowSeat is one seat for one show. It can only be AVAILABLE, LOCKED (held by one user until a deadline) or BOOKED. All fields are **private**; the only way to change a seat is through the transition methods, each of which checks that the move is legal.



Figure 3 - ShowSeat transitions

core/include/models/ShowSeat.hpp

```
class ShowSeat {
public:
    using Clock = std::chrono::steady_clock;

    ShowSeat(Seat seat, double basePrice, SeatStatus status = SeatStatus::AVAILABLE);

    const Seat& seat() const { return seat_; }
    const std::string& id() const { return seat_.id; }
    double basePrice() const { return basePrice_; }

    // Status as seen at `now`: a hold whose TTL has passed reads as AVAILABLE.
    SeatStatus statusAt(Clock::time_point now) const;
    bool isHeldBy(const std::string& userId, Clock::time_point now) const;
    Clock::time_point lockExpiry() const { return lockExpiry_; }
    // True until the hold is cleared, even if its TTL has already passed.
    bool hasHold() const { return status_ == SeatStatus::LOCKED; }

    bool canLock(Clock::time_point now) const;
    bool tryLock(const std::string& userId, Clock::time_point expiry, Clock::time_point now);
    bool unlock(const std::string& userId);
    bool confirm(const std::string& userId, Clock::time_point now);
    bool release();  // BOOKED -> AVAILABLE (booking cancelled)
    bool releaseIfExpired(Clock::time_point now);
```

Design point: an expired hold is treated as AVAILABLE immediately by `statusAt(now)`, even before the background thread cleans it up. Correctness never depends on the cleanup thread's timing; the thread only tidies memory.

4.2 Show - aggregate root and per-show lock

Show owns its seats and one mutex. Every seat change goes through Show, so a request for several seats is **all-or-nothing**: Phase 1 checks every seat, Phase 2 changes them all. Because nothing is changed until every check passes, there is never a partial state to roll back. Seats are found through a

hash map (O(1)) instead of a linear scan.

core/src/models/Show.cpp

```
bool Show::lockSeats(const std::vector<std::string>& seatIds, const std::string& userId,
                     Clock::duration ttl) {
    if (userId.empty() || ttl <= Clock::duration::zero()) {
        return false;
    }
    std::lock_guard<std::mutex> lock(mutex_);
    const auto seats = resolve(seatIds);
    const auto now = Clock::now();
    // Phase 1: check every seat. Phase 2: commit. No partial state is ever
    // visible, so there is nothing to roll back.
    const bool allFree = std::all_of(seats.begin(), seats.end(),
                                     [now](const ShowSeat* seat) { return seat->canLock(now); });
    if (seats.empty() || !allFree) {
        return false;
    }
    for (auto* seat : seats) {
        seat->tryLock(userId, now + ttl, now);
    }
    return true;
}
```

Why one lock per show instead of one per seat? A request only ever holds one show's lock, so deadlock is impossible by construction. Requests for different shows run fully in parallel. The critical section is a few microseconds, so finer per-seat locks would add lock-ordering complexity for no measurable gain.

4.3 SeatLockManager - Singleton registry

SeatLockManager is the single process-wide registry of live shows. Lookups take a **shared (reader) lock**, so many requests can find shows at the same time; only registering or removing a show takes the exclusive lock. It also runs a background thread that clears expired holds, woken by a condition variable so it stops instantly on shutdown.

core/src/services/SeatLockManager.cpp

```
SeatLockManager& SeatLockManager::getInstance() {
    // Function-local static: initialised once, thread-safe since C++11.
    static SeatLockManager instance;
    return instance;
}

std::shared_ptr<Show> SeatLockManager::getShow(const std::string& showId) const {
    std::shared_lock<std::shared_mutex> lock(registryMutex_);
    const auto it = shows_.find(showId);
    return it == shows_.end() ? nullptr : it->second;
}

bool SeatLockManager::lockSeats(const std::string& showId,
                                const std::vector<std::string>& seatIds,
                                const std::string& userId, Clock::duration ttl) {
    const auto show = getShow(showId);
    return show && show->lockSeats(seatIds, userId, ttl);
}
```

> **Interview point:** the function-local static is initialised exactly once and is thread-safe since C++11 ("magic statics"). Know the trade-off too: a Singleton is global state and harder to test. Here it is justified because seat holds must be shared by every request in the process; BookingService receives it by reference, so it is still injectable.

4.4 Pricing - Strategy + rule chain

Each pricing rule is a **Strategy** with one method, `apply(price, context)`. PricingEngine holds an ordered list of rules and runs them one after another. Day and hour are computed in the **theatre's timezone**, so the price is identical whether the server runs in India or in the cloud in UTC.

core/include/pricing/PricingStrategy.hpp

```
struct PricingContext {
    SeatType seatType{SeatType::REGULAR};
    std::tm showTime{};  // show start in the theatre's local time
};

// Strategy: one independent pricing rule. Rules are chained by PricingEngine,
// each receiving the price produced by the previous rule.
class PricingStrategy {
public:
    virtual ~PricingStrategy() = default;
    virtual double apply(double price, const PricingContext& context) const = 0;
};
```

core/src/pricing/*.cpp

```
double PeakHourPricing::apply(double price, const PricingContext& context) const {
    const int hour = context.showTime.tm_hour;
    return (hour >= 17 && hour < 22) ? price * 1.50 : price;
}

PricingEngine PricingEngine::withDefaultRules() {
    return PricingEngine({std::make_shared<SeatTypePricing>(),
                          std::make_shared<WeekendPricing>(),
                          std::make_shared<PeakHourPricing>()});
}

double PricingEngine::priceFor(double seatBasePrice, SeatType seatType,
                               std::chrono::system_clock::time_point showTime,
                               int utcOffsetMinutes) const {
    const PricingContext context{seatType, theatreLocalTime(showTime, utcOffsetMinutes)};
    double price = seatBasePrice;
    for (const auto& rule : rules_) {
        price = rule->apply(price, context);
    }
    return std::round(price * 100.0) / 100.0;
}
```

Rule	Condition	Effect	Example on Rs. 200 base
SeatTypePricing	Seat type	REGULAR x1.0, PREMIUM x1.25, VIP x1.5	VIP: 300
WeekendPricing	Saturday or Sunday (theatre time)	x1.2	Sat REGULAR: 240
PeakHourPricing	Starts 17:00 - 21:59 (theatre time)	x1.5	7 pm REGULAR: 300
All three	VIP, Saturday, 7 pm	200 x 1.5 x 1.2 x 1.5	540

Open/Closed in one sentence: to add a festival surcharge you write `class FestivalSurcharge : public PricingStrategy` and call `engine.addRule(...)`; no existing class changes. A unit test (`NewRuleCanBeAddedWithoutChangingExistingCode`) proves it. The same engine prices both the seat map and the booking, so the price shown is always the price charged.

4.5 Payments - Strategy + Factory

Every payment mode implements `PaymentStrategy::pay(amount)`. BookingService never names a concrete class: it asks the **Factory** for the strategy that matches the user's chosen mode. Payments are mocked (no real money moves); a real gateway would be one more strategy.

core/include/payment + core/src/payment/PaymentFactory.cpp

```
class PaymentStrategy {
public:
    virtual ~PaymentStrategy() = default;
    virtual bool pay(double amount) const = 0;
};

std::unique_ptr<PaymentStrategy> PaymentFactory::create(PaymentMode mode) {
    switch (mode) {
        case PaymentMode::CARD:
            return std::make_unique<CardPayment>();
        case PaymentMode::UPI:
            return std::make_unique<UpiPayment>();
        case PaymentMode::NET_BANKING:
            return std::make_unique<NetBankingPayment>();
    }
    throw std::invalid_argument("Unsupported payment mode");
}
```

4.6 Booking - the booking state machine

A Booking starts PENDING when the user proceeds to checkout. It becomes CONFIRMED when payment succeeds, CANCELLED when the user cancels, or EXPIRED when the seat hold runs out. CANCELLED and EXPIRED are final. The allowed moves live in one table, `canTransition`, instead of scattered if-statements.

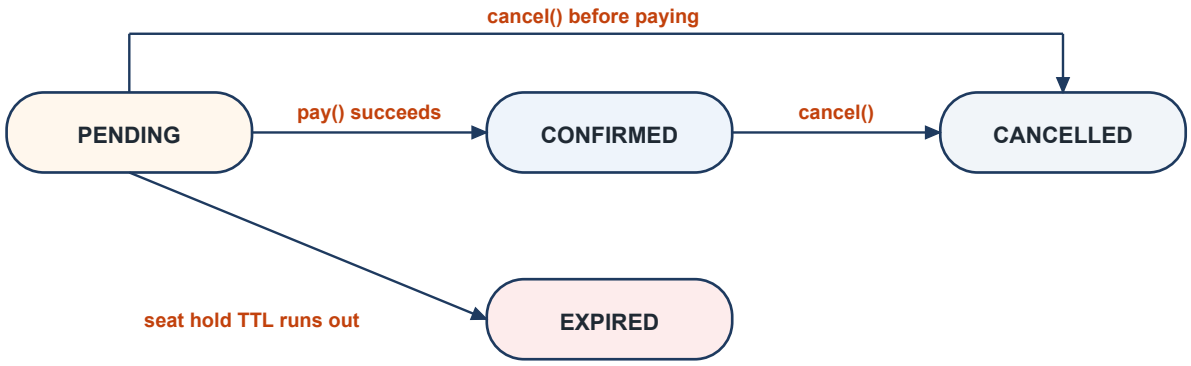


Figure 4 - Booking lifecycle

core/src/models/Booking.cpp

```
bool Booking::canTransition(BookingStatus from, BookingStatus to) {
    switch (from) {
        case BookingStatus::PENDING:
            return to == BookingStatus::CONFIRMED || to == BookingStatus::CANCELLED ||
                   to == BookingStatus::EXPIRED;
        case BookingStatus::CONFIRMED:
            return to == BookingStatus::CANCELLED;
        case BookingStatus::CANCELLED:
        case BookingStatus::EXPIRED:
            return false;  // final states
    }
    return false;
}

bool Booking::transitionTo(BookingStatus next) {
    if (!canTransition(status_, next)) {
        return false;
    }
    status_ = next;
    return true;
}
```

Business rule enforced by the model: a pending booking carries the expiry time of its seat hold. `confirmBooking` checks it *before* charging, so an expired booking is never paid for. If the hold runs out during payment, the charge is reversed and the booking moves to EXPIRED.

4.7 Refunds - RefundPolicy strategy

Cancelling a confirmed booking asks the injected **RefundPolicy** how much to return. The default **TieredRefundPolicy** gives a full refund 24 h or more before the show, 50% between 24 h and 2 h, and refuses cancellation in the last 2 hours. **FullRefundPolicy** (e.g. for a promotion) refunds everything until the show starts. Choosing a policy is configuration, not code change.

core/src/refund/TieredRefundPolicy.cpp

```
RefundDecision TieredRefundPolicy::evaluate(
    double paidAmount, std::chrono::system_clock::duration timeUntilShow) const {
    using std::chrono::hours;
    if (timeUntilShow >= hours(24)) {
        return {true, paidAmount, {}};
    }
    if (timeUntilShow >= hours(2)) {
        return {true, std::round(paidAmount * 50.0) / 100.0, {}};
    }
    return {false, 0.0, "Cancellation closes 2 hours before the show"};
}
```

4.8 BookingService - orchestrating the use cases

BookingService coordinates the flow: verify the user still holds the seats, price them, create the booking, take payment, confirm the seats, notify observers, and handle cancellations. Its collaborators are **injected** through the constructor, which is what makes pricing and refund rules swappable and the service unit-testable.

core/src/services/BookingService.cpp

```
BookingService::BookingService(SeatLockManager& seatLockManager,
                               NotificationService& notificationService,
                               const PricingEngine& pricingEngine,
                               std::shared_ptr<const RefundPolicy> refundPolicy)
    : seatLockManager_(seatLockManager),
      notificationService_(notificationService),
      pricingEngine_(pricingEngine),
      refundPolicy_(std::move(refundPolicy)) {
    if (!refundPolicy_) {
        throw std::invalid_argument("BookingService needs a refund policy");
    }
}

ConfirmBookingResult BookingService::confirmBooking(const std::string& bookingId,
                                                    PaymentMode mode) {
    Booking confirmedBooking;
    Payment payment;
    {
        std::lock_guard<std::mutex> lock(mutex_);
        const auto bookingIt = bookings_.find(bookingId);
        if (bookingIt == bookings_.end()) {
            return {false, {}, "Booking not found"};
        }
        Booking& booking = bookingIt->second;
        // Check the hold before charging, so an expired booking is never paid for.
        if (booking.isHoldExpired(Booking::Clock::now())) {
            booking.transitionTo(BookingStatus::EXPIRED);
            return {false, {}, kHoldExpiredReason};
        }
        if (!Booking::canTransition(booking.status(), BookingStatus::CONFIRMED)) {
            return {false, {}, "Booking is not pending"};
        }

        payment.id = "payment-" + std::to_string(++nextPaymentId);
        payment.bookingId = bookingId;
        payment.mode = mode;
        payment.amount = booking.totalAmount();
        payment.createdAt = std::chrono::system_clock::now();
        if (!PaymentFactory::create(mode)->pay(payment.amount)) {
            payment.status = PaymentStatus::FAILED;
            payments_[bookingId] = payment;
            return {false, payment, "Payment failed"};
        }
        if (!seatLockManager_.confirmSeats(booking.showId(), booking.seatIds(),
                                           booking.userId())) {
            // The hold ran out while paying: the charge is reversed.
            payment.status = PaymentStatus::REFUNDED;
            payments_[bookingId] = payment;
            booking.transitionTo(BookingStatus::EXPIRED);
            return {false, payment, kHoldExpiredReason};
        }

        payment.status = PaymentStatus::SUCCESS;
        payments_[bookingId] = payment;
        booking.transitionTo(BookingStatus::CONFIRMED);
        confirmedBooking = booking;
    }
    notificationService_.notify(confirmedBooking, "confirmed");
    return {true, payment, {}};
}
```

Lock ordering: BookingService's own mutex may be held while it calls into a Show, but a Show never calls back into BookingService. Locks are therefore always taken in the same order (service, then show), which rules out deadlock between them. Notifications are sent *after* releasing the lock, so a slow observer never blocks other bookings.

4.9 Notifications - Observer

When a booking is confirmed or cancelled, NotificationService notifies every subscribed **BookingObserver**. Email and SMS notifiers print to the console in this project; a real push or email service would be one more observer. The observer list is copied under the lock and the callbacks run outside it.

core/src/services/NotificationService.cpp

```
void NotificationService::notify(const Booking& booking, const std::string& event) const {
    std::vector<std::shared_ptr<BookingObserver>> observers;
    {
        std::lock_guard<std::mutex> lock(mutex_);
        observers = observers_;
    }
    for (const auto& observer : observers) {
        observer->onBookingEvent(booking, event);
    }
}
```

4.10 BookingEngine - Facade over everything

Node.js should not know about ShowSeat or PricingEngine. BookingEngine is a **Facade**: a handful of methods that take and return JSON strings. The Node-API addon wraps each one, and the backend calls them like normal JavaScript functions.

core/include/BookingEngine.hpp

```
// Facade: the single JSON-in / JSON-out entry point used by the Node.js addon.
// It hides the models, SeatLockManager, BookingService, pricing and payments.
class BookingEngine {
public:
    explicit BookingEngine(ShowSeat::Clock::duration lockTtl = std::chrono::seconds(300),
                           std::chrono::milliseconds expiryInterval = std::chrono::seconds(1),
                           std::shared_ptr<const RefundPolicy> refundPolicy = nullptr,
                           PricingEngine pricingEngine = PricingEngine::withDefaultRules());
    ~BookingEngine();

    BookingEngine(const BookingEngine&) = delete;
    BookingEngine& operator=(const BookingEngine&) = delete;

    std::string loadShow(const std::string& json);
    std::string getSeatMap(const std::string& json);
    std::string lockSeats(const std::string& json);
    std::string unlockSeats(const std::string& json);
    std::string createBooking(const std::string& json);
    std::string pay(const std::string& json);
    std::string cancel(const std::string& json);
    std::string releaseBookedSeats(const std::string& json);
    std::string quoteRefund(const std::string& json);
    std::string releaseExpired(const std::string& json = "{}");

    NotificationService& notifications();
```

Engine call	Input	Output
loadShow	showId, startTime, basePrice, utcOffsetMinutes?, seats[]	{ ok }
getSeatMap	showId	[{ seatId, row, number, type, status, price }]
lockSeats / unlockSeats	showId, seatIds[], userId	{ ok, expiresAt? }

Engine call	Input	Output
createBooking	showId, seatIds[], userId	{ ok, bookingId, totalAmount }
pay	bookingId, mode	{ ok, status, paymentId? }
cancel	bookingId	{ ok, refundAmount }
quoteRefund	showId, amount	{ ok, refundAmount, reason? }
releaseBookedSeats	showId, seatIds[]	{ ok }
releaseExpired	-	{ released, expiredBookings }

5. Concurrency design

The central problem of a ticketing system: **two people click the same seat at the same moment**. Exactly one must win, the loser must see a clear failure, nobody may end up with half of their seats, and the system must never freeze.

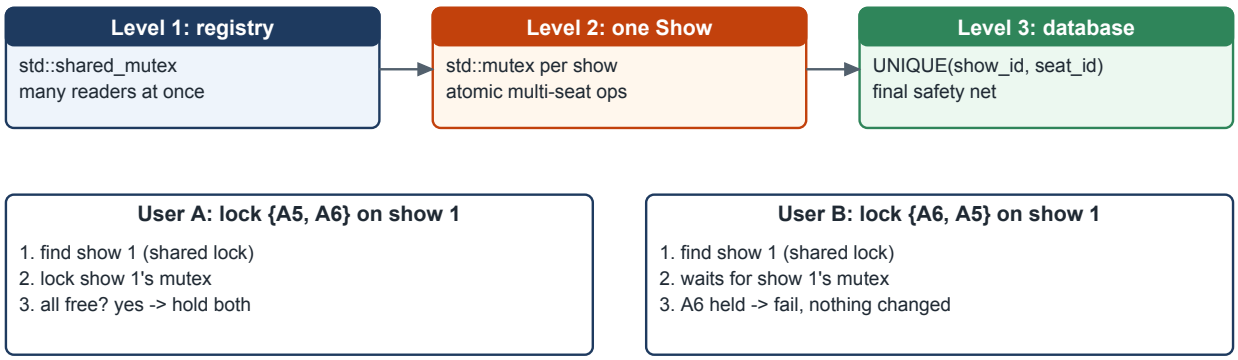


Figure 5 - Three levels of protection, and two users racing for the same seats

Requirement	How the design meets it
Only one winner per seat	All seat checks and changes for a show happen under that show's mutex, so two requests are processed one after the other.
All-or-nothing for multiple seats	Check every seat first, change them only if all are free. A failed request changes nothing.
No deadlock	A request holds at most one show lock at a time; the registry lock is released before the show lock is taken; service-then-show is the only nesting order.
Different shows do not block each other	Each show has its own mutex; the registry uses a reader/writer lock so lookups run in parallel.
Abandoned holds come back	Each hold has a deadline (steady_clock, immune to clock changes). Expired holds read as AVAILABLE immediately; a background thread clears them.
Safety net across processes	UNIQUE (show_id, seat_id) in Postgres rejects a second confirmed booking even if the in-memory layer were bypassed.

Verified, not just claimed: a test starts 50 threads that race for the same five seats and asserts exactly one wins; another locks seats in opposite orders to show there is no deadlock. The whole suite was run 50 times in a row and under **ThreadSanitizer** (a data-race detector) with zero reports.

6. End-to-end booking flow

What happens from the moment the user clicks *Proceed* until the booking is saved. Solid arrows are calls, dashed arrows are responses.

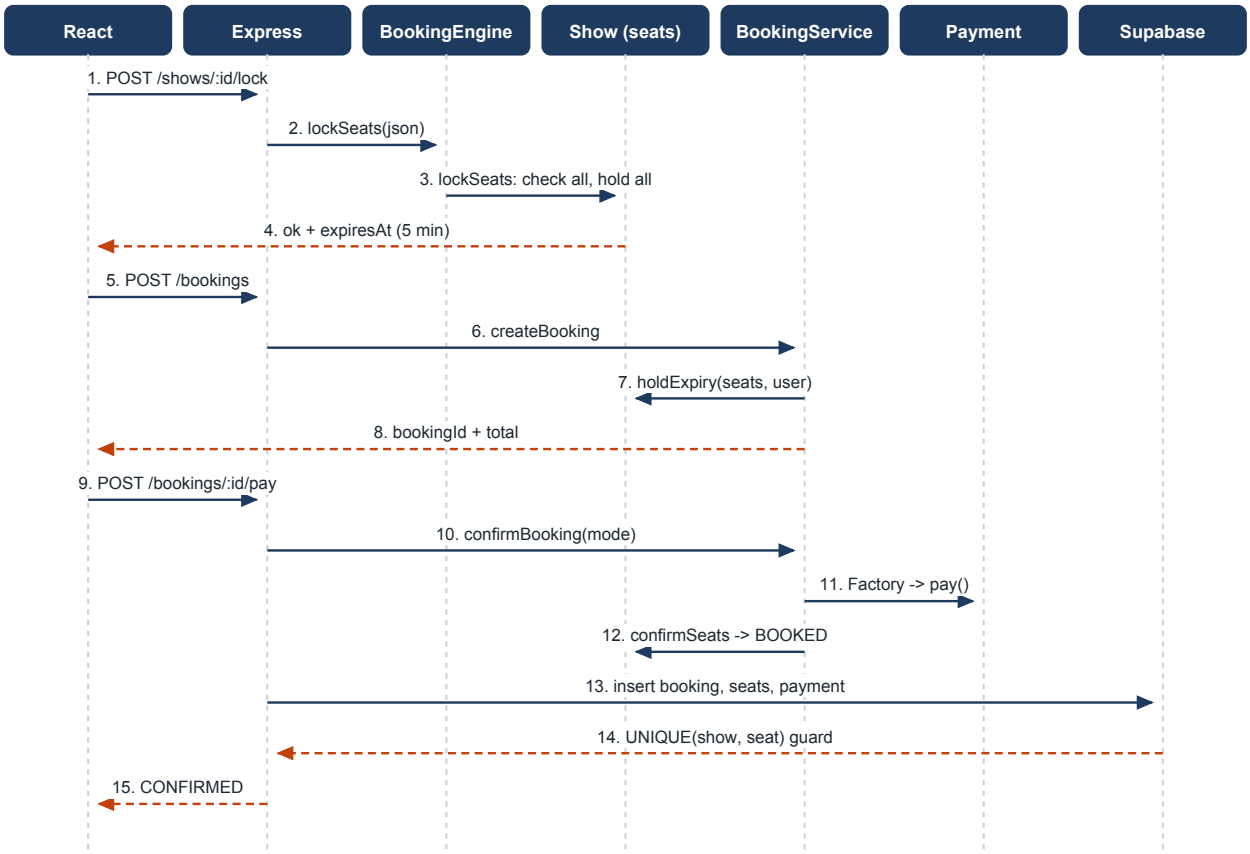


Figure 6 - Lock, book and pay sequence

- **Steps 1-4:** seats are held for the TTL; the frontend shows a countdown using `expiresAt`.
- **Steps 5-8:** the booking is created only if the user still holds every seat; its total comes from PricingEngine, the same engine that priced the seat map.
- **Steps 9-12:** payment runs through the Factory-created strategy, then the seats become BOOKED.
- **Steps 13-15:** the backend persists the booking. If the database write fails, it **compensates**: frees the seats in the engine and deletes partial rows.

7. Backend and persistence

Concern	Approach	File
Hydration	First request for a show loads its seats and confirmed bookings from Postgres into the engine. Concurrent first requests share one in-flight load.	backend/src/services/show.service.js
Persist after confirm	Booking, booking seats and payment rows are written only after the engine confirms payment.	backend/src/services/booking.service.js
Compensation	If a write fails, seats are released in the engine and partial rows removed (a small saga).	booking.service.js - pay()
Cancellation	Engine quotes the refund (RefundPolicy), database is updated, refund is recorded as its own payment row, seats are freed - also after a restart.	booking.service.js - cancel()
Pay/cancel race	A booking is marked PAYING while it is being saved; cancel is rejected during that window.	booking.service.js
Auth	bcrypt password hashes, JWT bearer tokens, users can only see their own bookings.	backend/src/services/auth.service.js
Security	Row Level Security on every table; only the server-side service-role key can read or write data.	db/schema.sql

Database schema

```
users(id, name, email UNIQUE, phone, password_hash)
cities(id, name)  theatres(id, city_id, name, address)  screens(id, theatre_id, name)
seats(id, screen_id, row_label, seat_number, seat_type)
movies(id, title, duration_mins, language, genre, poster_url)
shows(id, movie_id, screen_id, start_time, end_time, base_price)
bookings(id, user_id, show_id, status, total_amount, created_at)
booking_seats(booking_id, seat_id, show_id, price)   UNIQUE (show_id, seat_id)
payments(id, booking_id, mode, amount, status, created_at)   -- refunds are extra rows
```

8. SOLID principles in this code

Principle	Where to point in the code
Single responsibility	ShowSeat only knows seat state; Show only guards its seats; PricingEngine only prices; BookingService only orchestrates; notifiers only notify.
Open/closed	New pricing rule, payment mode, refund policy or notifier = new class; existing classes are untouched (tested for pricing).
Liskov substitution	Any PricingStrategy, PaymentStrategy, RefundPolicy or BookingObserver can replace another; callers rely only on the interface.
Interface segregation	Each interface has one method (apply, pay, evaluate, onBookingEvent); no class implements methods it does not need.
Dependency inversion	BookingService depends on PricingEngine and RefundPolicy abstractions passed into its constructor, not on concrete rules.

9. Testing

Suite	Tests	What it proves
SeatLockManagerTests	5	One winner among 2 and 50 racing threads; opposite seat orders do not deadlock; a failed multi-seat lock changes nothing; the background thread clears expired holds.
BookingEngineTests	5	Full flow lock -> book -> pay -> cancel; pending cancel; expired holds cannot be booked; seat-map price equals charged total; releasing seats of an untracked booking.
BookingLifecycleTests	5	Legal and illegal booking transitions; tiered and full refund policies; injected policy decides refund; pending booking expires with its hold.
PricingAndPaymentTests	5	Each rule in isolation; theatre time vs server time; adding a rule without changing code; factory and payment strategies.
Backend (node:test + Supertest)	5	HTTP flow end to end with a mocked database, lock race over HTTP, compensation on write failure, price consistency, cancel after restart.

C++ tests also pass with the process timezone set to New York, India and UTC, which proves pricing does not depend on the server's clock settings.

10. Trade-offs and scaling

Decision	Benefit	Cost / next step
Seat holds in memory	Microsecond locks, no database load for browsing	Single process only; a restart drops active holds (confirmed bookings are safe).
One mutex per show	Simple, deadlock-free, parallel across shows	Very large shows serialise on one lock; could shard by seat block if ever needed.
Singleton lock manager	One shared view of holds	Global state; mitigated by passing it by reference into BookingService.
Mocked payments	Deterministic flow and tests	Real gateway = new PaymentStrategy + idempotency key + webhook confirmation.
Database UNIQUE guard	Final protection against double booking	Detects conflicts late (after payment); the in-memory lock prevents them early.

How to scale to many servers

- Move seat holds to **Redis** with `SET key value NX PX ttl` (atomic hold with expiry) or a Lua script for multi-seat all-or-nothing.
- Or route all requests for one show to the same server (consistent hashing on showId), keeping today's in-memory design per shard.
- Use database transactions with `SELECT ... FOR UPDATE SKIP LOCKED` as an alternative when traffic is moderate.
- Publish booking events to a queue (Kafka/SQS) and let notification workers consume them - the Observer pattern, distributed.

11. Interview guide

2-minute pitch

> "I built a movie ticket booking system whose core is a C++17 booking engine. The hard problem is two users grabbing the same seat, so each seat for a show is a small state machine - available, locked, booked - and a Show object guards its seats under one mutex, checking every requested seat before changing any, so multi-seat holds are all-or-nothing and can't deadlock. Holds expire after five minutes. Pricing, payment and refund rules are Strategies - pricing is a rule chain, so I can add a surcharge without touching existing code - payments come from a Factory, notifications use Observer, and a Facade exposes everything to Node.js as JSON. Bookings have their own state machine so expired bookings can't be paid. It's covered by concurrency tests, including 50 threads racing for the same seats, and runs clean under ThreadSanitizer."

10-minute walkthrough order

- **1. Requirements** (1 min): browse, hold seats, pay within a time limit, cancel with refund, no double booking.
- **2. Domain model** (2 min): physical Seat vs ShowSeat; Show as aggregate root; Booking and Payment. Draw Figure 2.
- **3. Concurrency** (3 min): ShowSeat states, two-phase lock in Show, why one lock per show, TTL expiry, database guard. Draw Figure 5.
- **4. Extensibility** (2 min): pricing rule chain, payment factory, refund policy, observers - show how a new rule is added.
- **5. Lifecycle and failures** (1 min): booking state machine, expired holds, compensation when the database write fails.
- **6. Trade-offs and scale** (1 min): single process today, Redis or sharding tomorrow.

Likely questions and strong answers

Q. How do you prevent two users booking the same seat?

All changes to a show's seats happen under that show's mutex, with a check-then-commit two-phase update. One request finishes before the other starts, so the second sees the seat as LOCKED and fails. The database UNIQUE constraint is the final safety net.

Q. Can your locking deadlock?

No. A request holds at most one show lock; the registry lock is released before taking it; and the only nested order is BookingService then Show, never the reverse.

Q. What if a user locks seats and closes the browser?

Every hold has a deadline. After it passes the seat reads as AVAILABLE immediately and a background thread clears it. The frontend also releases holds on page exit.

Q. Why is SeatLockManager a Singleton? Isn't that an anti-pattern?

All requests in the process must share one view of seat holds and one expiry thread. To limit the downsides it is passed by reference into services rather than accessed globally everywhere.

Q. How would you add a festival surcharge?

Write a new PricingStrategy and add it to PricingEngine. No existing class changes - there is a unit test that does exactly this.

Q. Why Strategy and not if/else for payment?

Each mode has different behaviour and will grow (gateways, retries). Strategy isolates each one; the Factory keeps construction in one place.

Q. What happens if payment succeeds but saving to the database fails?

The backend compensates: it frees the seats in the engine and deletes any partial rows. In production you would also refund through the gateway.

Q. What if the hold expires while the user is paying?

The booking checks its hold expiry before charging. If it expires mid-payment, seat confirmation fails, the charge is reversed and the booking moves to EXPIRED.

Q. Why steady_clock for expiry?

It never jumps when the system clock is changed (NTP, daylight saving). system_clock is used only for real show times.

Q. How is the refund decided?

A RefundPolicy strategy: full refund 24 h or more before the show, 50% from 24 h to 2 h, no cancellation after that. A different policy is injected, not coded in.

Q. How would you scale to multiple servers?

Move holds to Redis (SET NX PX for atomic hold + TTL) or shard shows across servers by showId. Keep the database UNIQUE constraint as the final guard.

Q. Why keep holds in memory instead of the database?

Holds are short-lived and very frequent; memory gives microsecond locks with no database load. Only confirmed bookings, which must survive restarts, go to Postgres.

Q. How did you test concurrency?

Tests race 2 and 50 threads for the same seats, lock in opposite orders, and check the background expiry. The suite was repeated 50 times and run under ThreadSanitizer.

Q. Why is price computed in theatre time?

Weekend and peak hour belong to the theatre's local clock. Using the server's timezone would charge different prices on a laptop in India and a cloud server in UTC.

Q. What would you improve next?

Repository interfaces for persistence, idempotency keys for payments, a maximum seats per booking, typed events, and Redis-backed holds for horizontal scaling.

12. Appendix

File index (C++ core)

File (under core/)	Purpose
include/BookingEngine.hpp	Facade; JSON API used by Node.js
include/enums/Enums.hpp	All enums
include/models/ShowSeat.hpp	Seat state machine (private state)
include/models/Show.hpp	Aggregate root, per-show mutex, two-phase seat ops
include/models/Booking.hpp	Booking state machine
include/services/SeatLockManager.hpp	Singleton registry + expiry thread
include/services/BookingService.hpp	Use-case orchestration, injected collaborators
include/services/NotificationService.hpp	Observer subject
include/pricing/PricingStrategy.hpp	Pricing Strategy interface + context
include/pricing/PricingEngine.hpp	Rule chain, theatre-time pricing
include/payment/PaymentFactory.hpp	Factory for payment strategies
include/refund/RefundPolicy.hpp	Refund Strategy interface
include/observer/BookingObserver.hpp	Observer interface
src/bindings/addon.cpp	Node-API wrapper
src/main.cpp	Runnable demo of the whole flow

Running the project

```
# C++ engine: build and run tests / demo
cmake -S core -B core/build && cmake --build core/build
./core/build/bookmyshow_tests
./core/build/bookmyshow_sample

# Backend (needs backend/.env with SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY)
cd backend && npm install && npm run build:core && npm start      # http://localhost:4000

# Frontend
cd frontend && npm install && npm run dev                         # http://localhost:5173
```